

**LAPORAN PRAKTIKUM
INTERNET OF THINGS (IOT) II**

Modul 1 dan Modul 2



Disusun Oleh :

Nama : Naila Rahma Chintya Aulya

NPM : 25782020

Kelas : 3A

Dosen Pengampu : Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

Andi Saptono, A.Md

**JURUSAN TEKNOLOGI INFORMASI
PROGRAM STUDI TEKNOLOGI REKAYASA INTERNET
POLITEKNIK NEGERI LAMPUNG**

2026/2027

Modul 1

Praktikum 1.5.1

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



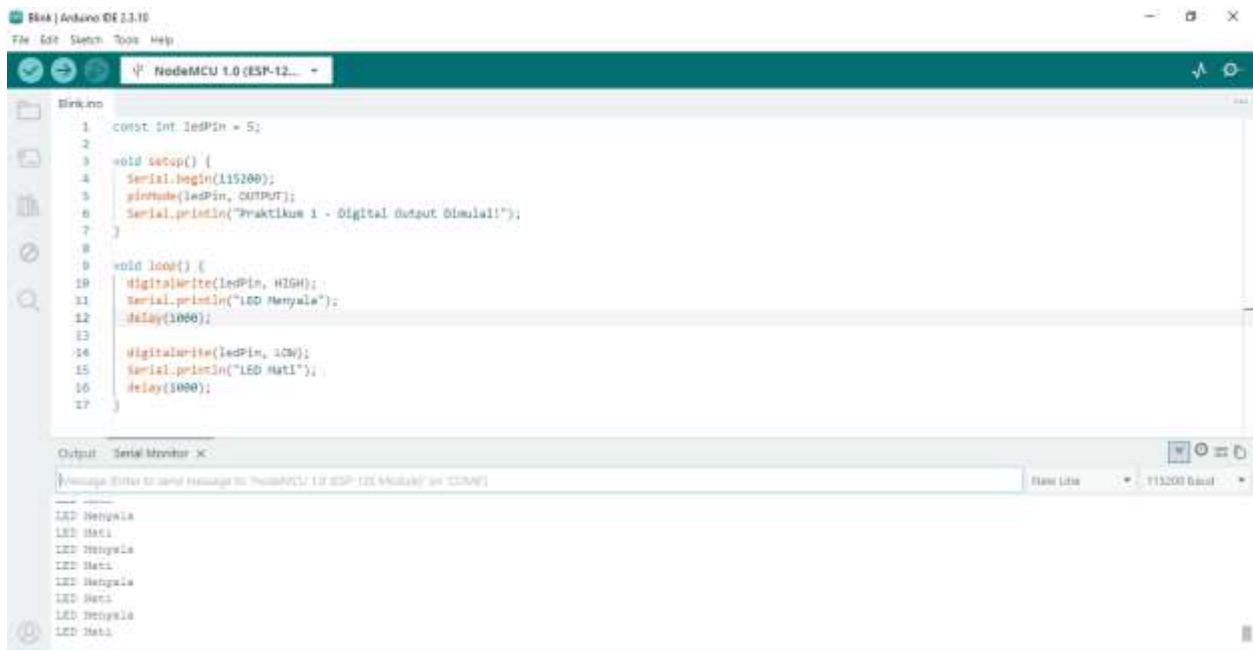
Langkah Kerja Rangkaian:

1. Pasang NodeMCU pada *breadboard*.
2. Pasang LED pada baris kosong di *breadboard*. Perhatikan kakinya:
 - Kaki yang lebih panjang adalah Anoda (Positif).
 - Kaki yang lebih pendek (dan terdapat sisi datar di tepi plastik kepala LED) adalah Katoda (Negatif).

3. Hubungkan pin D1 (GPIO 5) dari NodeMCU ke kaki Anoda LED menggunakan kabel *jumper*.
4. Hubungkan kaki Katoda LED ke salah satu ujung Resistor 220 Ohm.
5. Hubungkan ujung Resistor lainnya ke jalur GND pada NodeMCU. (*Catatan: Resistor berfungsi sebagai pembatas arus agar LED tidak terbakar*).

Langkah Kerja Pemrograman:

1. Buka Arduino IDE, klik File > New Sketch.
2. Tuliskan kode program berikut:



```
1 const int ledPin = 5;
2
3 void setup() {
4   Serial.begin(115200);
5   pinMode(ledPin, OUTPUT);
6   Serial.println("Praktikum 1 - Digital Output Simulasi");
7 }
8
9 void loop() {
10  digitalWrite(ledPin, HIGH);
11  Serial.println("LED Menyala");
12  delay(1000);
13
14  digitalWrite(ledPin, LOW);
15  Serial.println("LED Mati");
16  delay(1000);
17 }
```

Output - Serial Monitor X

Message (Click to send message to "NodeMCU 1.0 ESP-12E Module" on COM9)

LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati
LED Menyala
LED Mati

3. Klik tombol Upload (ikon panah ke kanan). Tunggu hingga muncul notifikasi *Done uploading*.
4. Buka Serial Monitor (ikon kaca pembesar di sudut kanan atas IDE).
5. Ubah pengaturan *baud rate* pada pojok kanan bawah Serial Monitor menjadi 115200 baud.
6. Amati hasilnya: LED fisik berkedip setiap 1 detik selaras dengan teks status yang dicetak pada Serial Monitor.

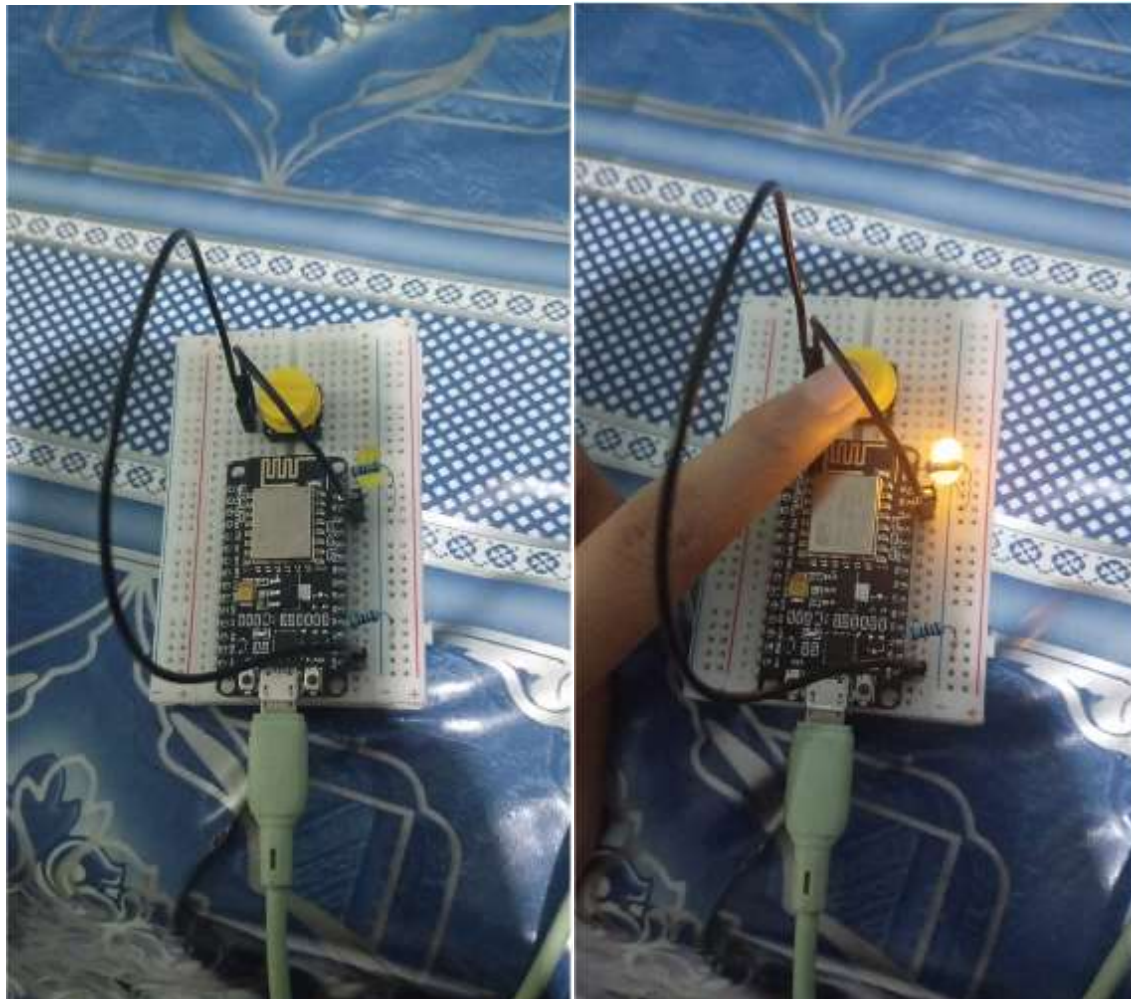
Penjelasan Singkat Kode:

- `setup()`: Blok fungsi yang dieksekusi hanya satu kali saat NodeMCU dinyalakan atau di-reset. Biasanya digunakan untuk pengaturan awal.

- **loop():** Blok fungsi yang dieksekusi secara berulang-ulang (terus menerus) selama NodeMCU memiliki daya.
- **pinMode(ledPin, OUTPUT):** Memberi instruksi bahwa pin D1 (GPIO 5) akan digunakan sebagai jalur keluar (*Output*) tegangan.
- **digitalWrite(ledPin, HIGH):** Memberikan tegangan 3.3V secara digital ke pin tersebut, sehingga arus mengalir dan LED menyala.
- **delay(1000):** Menghentikan sementara eksekusi program (jeda) selama 1000 milidetik (1 detik).

1.5.2 Praktikum 2: *Digital Input* (Membaca *Push Button*)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor *pull-down* eksternal.

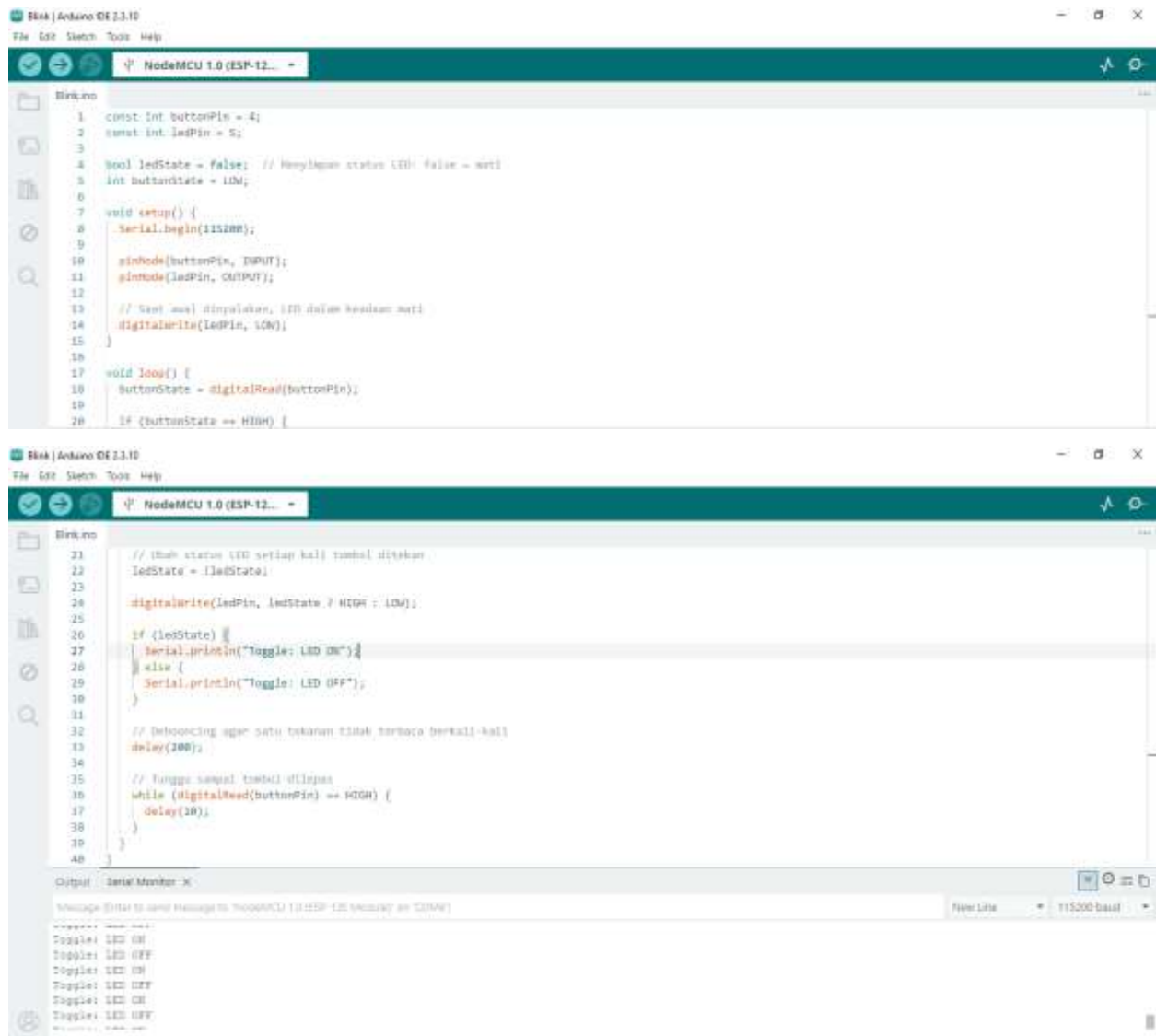


Langkah Kerja Rangkaian:

1. Biarkan rangkaian LED pada pin D1 dari Praktikum 1 tetap terpasang.
2. Pasang sebuah *push button* pada *breadboard*, melintasi garis pemisah/parit tengah.
3. Hubungkan kaki pertama *push button* ke pin 3V3 (3.3V) pada NodeMCU.
4. Hubungkan kaki kedua *push button* ke pin D2 (GPIO 4) NodeMCU.
5. Pada kaki kedua yang sama, pasang sebuah Resistor 10k Ohm, lalu hubungkan ujung lain dari resistor tersebut ke jalur GND NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*) pada Arduino IDE.
2. Tuliskan kode program berikut:



```
1 const int buttonPin = 4;
2 const int ledPin = 5;
3
4 bool ledState = false; // Mengatur status LED: false = mati
5 int buttonState = LOW;
6
7 void setup() {
8   Serial.begin(115200);
9
10  pinMode(buttonPin, INPUT);
11  pinMode(ledPin, OUTPUT);
12
13  // Saat awal dimulainya, LED dalam keadaan mati
14  digitalWrite(ledPin, LOW);
15 }
16
17 void loop() {
18   buttonState = digitalRead(buttonPin);
19
20   if (buttonState == HIGH) {
```

```
21 // (Saat status LED setiap kali tombol ditekan)
22 ledState = !ledState;
23
24 digitalWrite(ledPin, ledState ? HIGH : LOW);
25
26 if (ledState) {
27   Serial.println("Toggle: LED ON");
28 } else {
29   Serial.println("Toggle: LED OFF");
30 }
31
32 // Delooping agar satu tekanan tidak terbaca berkali-kali
33 delay(200);
34
35 // Fungsi sampai tombol dilepas
36 while (digitalRead(buttonPin) == HIGH) {
37   delay(10);
38 }
39 }
40 }
```

Output Serial Monitor X

Message (Serial to serial monitor to: NodeMCU (3.3V 128KB) on: USB)

Power: 1.10V 115200 baud

```
Toggle: LED ON
Toggle: LED OFF
Toggle: LED ON
Toggle: LED OFF
Toggle: LED ON
Toggle: LED OFF
```

3. Upload program ke dalam mikrokontroler.
4. Buka Serial Monitor.
5. Uji fungsionalitasnya: Saat tombol ditekan dan ditahan, LED akan menyala terang. Saat tombol dilepaskan, LED langsung padam.

Penjelasan Singkat Kode:

- **pinMode(buttonPin, INPUT):** Menetapkan pin D2 (GPIO 4) sebagai jalur masuk (*Input*) untuk mendeteksi / membaca tegangan dari luar.
- **digitalRead(buttonPin):** Fungsi yang bertugas membaca status tegangan pada pin saat ini. Hasilnya akan berupa HIGH (ada tegangan) atau LOW (tidak ada tegangan).
- **if (buttonState == HIGH):** Blok logika *conditional*. Karena kita menggunakan rangkaian resistor *pull-down*, menekan tombol akan mengalirkan arus 3.3V ke pin mikrokontroler. Sehingga, status terbaca HIGH dan kondisi if terpenuhi (LED dinyalakan).

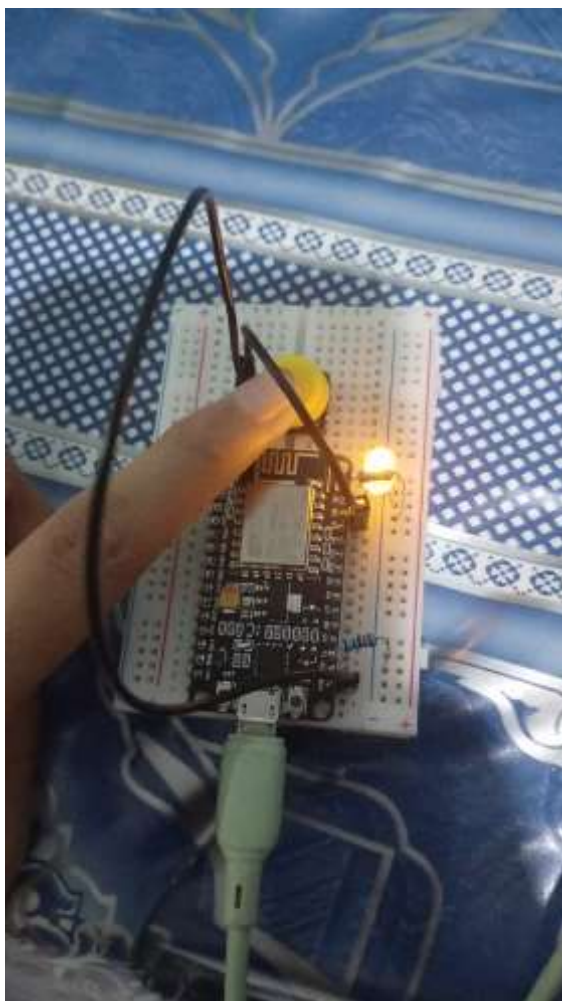
1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar *Toggle (Latching)*

- **Tujuan:** Mampu mengimplementasikan logika pelacakan status (*state tracking*) dan membedakan antara perilaku tombol *momentary* dengan *latching*.
- **Tingkat Kesulitan:** Menengah
- **Instruksi:**
 - i. Gunakan konfigurasi *hardware* yang persis sama dengan Praktikum 2 (Rangkaian *Pull-Down*).
 - ii. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali *lalu dilepas*, LED menyala dan tetap menyala.

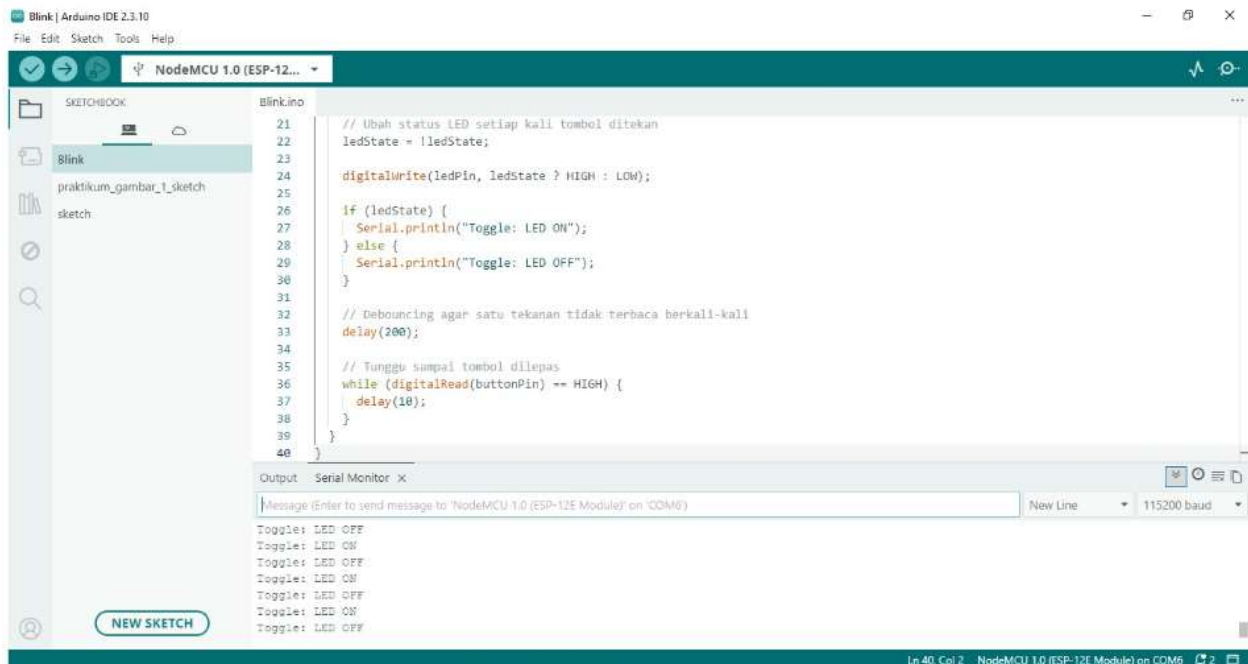
- Jika tombol ditekan 1 kali lagi *lalu dilepas*, LED mati dan tetap mati.
- iii. **Petunjuk:** Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (*debouncing* pada tombol fisik).
- **Kriteria Keberhasilan:**
 - Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan *satu kali tekanan singkat* (bukan ditahan).
 - Tidak ada fenomena *flickering* (lampu berkedip acak atau gagal mati) saat tombol ditekan.
 - Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan *state* dan *debounce* bekerja.





Blink.ino

```
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  bool ledState = false; // Menyimpan status LED: false = mati
5  int buttonState = LOW;
6
7  void setup() {
8      Serial.begin(115200);
9
10     pinMode(buttonPin, INPUT);
11     pinMode(ledPin, OUTPUT);
12
13     // Saat awal dinyalakan, LED dalam keadaan mati
14     digitalWrite(ledPin, LOW);
15 }
16
17 void loop() {
18     buttonState = digitalRead(buttonPin);
19
20     if (buttonState == HIGH) {
```

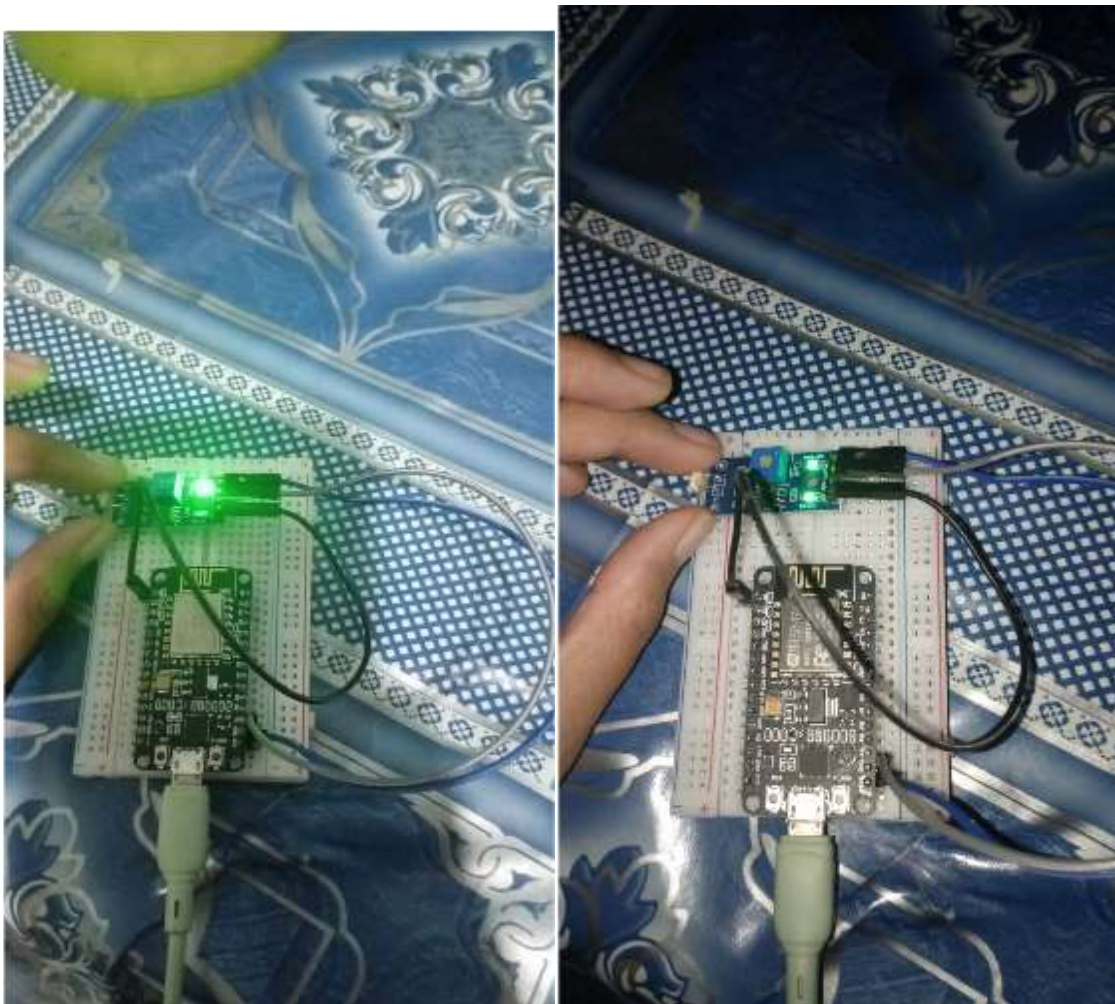


Modul 2

2.5 Praktikum

2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC

Sensor LDR akan menurunkan hambatannya (resistansi) saat terpapar cahaya terang, dan meningkatkan resistansinya saat gelap. Untuk menerjemahkan perubahan resistansi ini menjadi variasi tegangan yang bisa diukur oleh pin A0, kita menggunakan sirkuit pembagi tegangan (*Voltage Divider*).



Langkah Kerja Rangkaian:

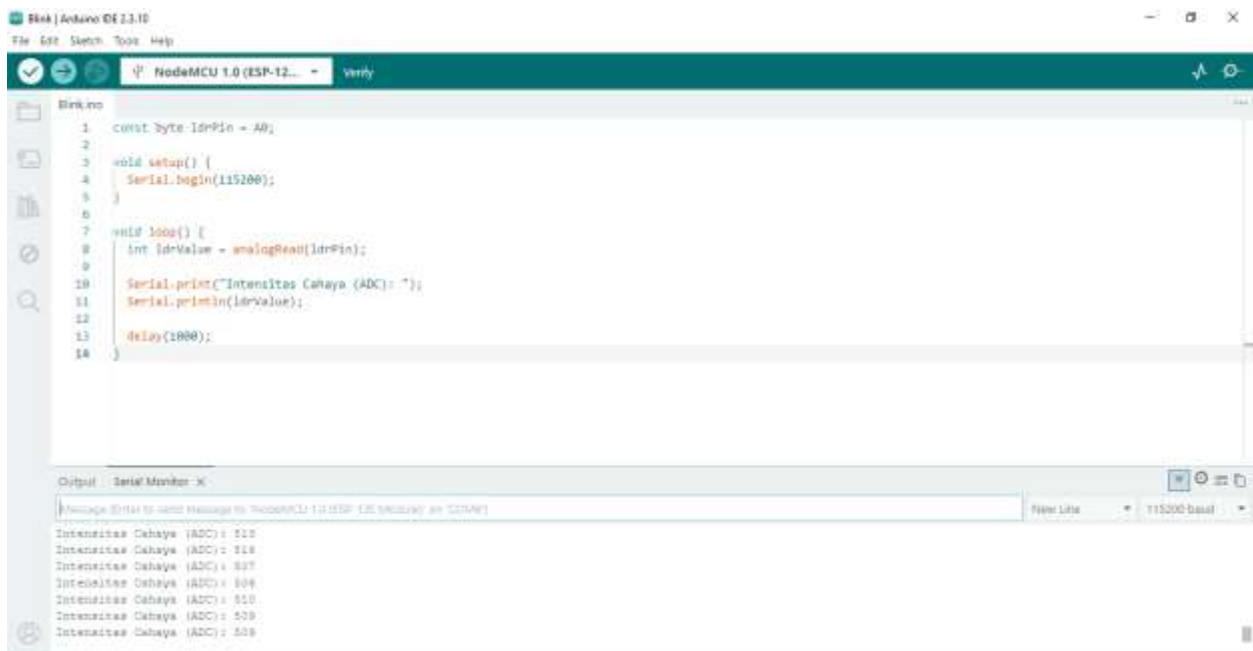
1. Tancapkan satu kaki LDR ke lubang *breadboard* yang terhubung langsung ke pin 3V3 (3.3 Volt) NodeMCU.
2. Tancapkan kaki LDR yang satunya ke sebuah jalur parit kosong di *breadboard*.

3. Hubungkan parit LDR tersebut dengan pin A0 menggunakan sebuah kabel *jumper*.
4. Pada parit LDR yang sama (sejajar dengan kabel ke A0), pasang salah satu kaki Resistor 10k Ohm.
5. Hubungkan ujung hulu Resistor 10k Ohm tersebut ke rel GND.

(Logika Rangkaian: Saat cahaya terang, resistansi LDR kecil sehingga arus 3.3V membanjiri A0, menghasilkan nilai ADC besar mendekati 1023. Saat gelap, resistansi LDR sangat besar menahan arus, sehingga pin A0 'tertarik' oleh resistor 10k ke arah GND, menghasilkan nilai ADC kecil mendekati 0).

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program berikut:



The screenshot shows the Arduino IDE interface. The main editor window displays a C++ sketch for an LDR sensor. The code defines a pin constant, initializes the serial port at 115200 baud, and reads the ADC value from the LDR pin in a loop, printing it to the serial monitor every 2000 milliseconds. The serial monitor at the bottom shows the output of the program, displaying the ADC values for 'Intensitas Cahaya'.

```
1 const byte ldrPin = A0;
2
3 void setup() {
4   Serial.begin(115200);
5 }
6
7 void loop() {
8   int ldrValue = analogRead(ldrPin);
9
10  Serial.print("Intensitas Cahaya (ADC): ");
11  Serial.println(ldrValue);
12
13  delay(2000);
14 }
```

Output Serial Monitor: K

Message (HTML to send Message to: NodeMCU 1.0 (ESP-12E Module) on COM4)

Intensitas Cahaya (ADC): 518
Intensitas Cahaya (ADC): 518
Intensitas Cahaya (ADC): 527
Intensitas Cahaya (ADC): 509
Intensitas Cahaya (ADC): 510
Intensitas Cahaya (ADC): 509
Intensitas Cahaya (ADC): 509

3. Upload program. Buka Serial Monitor (pastikan setelan 115200 baud).
4. Instruksi Pengujian: Sorot LDR dengan lampu *flash* ponsel dan amati angka yang membesar. Lalu, tutup bagian atas LDR rapat-rapat dengan jari dan amati penurunannya.

Penjelasan Singkat Kode:

- `const byte ldrPin = A0;` Mendeklarasikan pin ADC. Tipe byte digunakan (menggantikan int) untuk menghemat memori (*RAM*), karena ID pin tidak akan melebihi angka 255.
- `analogRead(ldrPin);` Fungsi bawaan mikrokontroler untuk membaca level tegangan analog murni di pin A0 dan mengonversinya menjadi data integer 10-bit (berkisar antara 0 hingga 1023).

2.5.2 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)

Sensor DHT menggunakan protokol komunikasi digital *single-wire*. Artinya, seluruh data (suhu dan kelembapan) ditransmisikan melalui satu kabel data saja. Protokol ini sangat bergantung pada waktu (*timing*). Mikrokontroler harus mengirimkan perintah "minta data" dengan durasi *HIGH* dan *LOW* yang sangat presisi. Setelah itu, sensor akan mengirimkan kembali serangkaian pulsa digital yang panjangnya spesifik menunjukkan nilai suhu dan kelembapan.



Langkah Kerja Rangkaian:

1. Hubungkan pin VCC / + / 3V3 pada DHT ke pin 3V3 NodeMCU.
2. Hubungkan pin GND / - pada DHT ke pin GND NodeMCU.
3. Hubungkan pin DATA / OUT / S pada DHT ke pin D7 (GPIO 13) pada NodeMCU.

Langkah Kerja Pemrograman:

1. Buat berkas baru (*New Sketch*).
2. Tuliskan kode program berikut:

```
Blinking
1  #include <DHT.h>
2
3  const byte dhtPin = 13;
4  #define DHTTYPE DHT22
5
6  DHT dht(dhtPin, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     dht.begin();
11
12     Serial.println("Memulai Sensor Lingkungan...");
13 }
14
15 void loop() {
16     delay(2500);
17
18     float temp = dht.readTemperature();
19     float hum = dht.readHumidity();
20
21     if (isnan(temp) || isnan(hum)) {
22         Serial.println("Gagal membaca data dari sensor DHT!");
23         return;
24     }
25
26     Serial.print("Suhu: ");
27     Serial.print(temp);
28     Serial.print(" Celcius | Kelembapan: ");
29     Serial.print(hum);
30     Serial.println(" %");
31 }
```

Output: Serial Monitor: X

Message (Enter to send message to "YanDeMCU_1.0 (ESP-12E Module)" on "COM1")

New Line 115200 baud

```
Suhu: 38.70 Celcius | Kelembapan: 38.70 %
Suhu: 38.70 Celcius | Kelembapan: 38.60 %
Suhu: 38.70 Celcius | Kelembapan: 38.50 %
Suhu: 38.80 Celcius | Kelembapan: 38.60 %
Suhu: 38.80 Celcius | Kelembapan: 38.00 %
Suhu: 38.80 Celcius | Kelembapan: 38.00 %
Suhu: 38.90 Celcius | Kelembapan: 38.60 %
```

3. Upload program dan buka Serial Monitor. Anda akan melihat data suhu lingkungan dan persentase kelembapan udara dicetak setiap 2.5 detik.

Penjelasan Singkat Kode:

- **#include <DHT.h>:** Mengimpor fungsi-fungsi dari *library* Adafruit DHT yang baru saja Anda instal.
- **#define DHTTYPE DHT11:** Mendefinisikan konstanta preprosesor untuk tipe sensor. (Ubah menjadi DHT22 jika sensor Anda berwarna putih).
- **DHT dht(dhtPin, DHTTYPE);:** Membentuk "objek" pengendali bernama dht yang mengingat pin data dan jenis sensor yang digunakan.
- **float temp = dht.readTemperature();:** Perintah ini bertugas membaca suhu dan menyimpannya dalam variabel float (karena mengandung angka desimal).
- **isnan(temp):** *Is Not a Number*. Fungsi matematika yang bertindak sebagai sistem perlindungan (*error handling*). Jika kabel sensor putus, *library* akan menghasilkan

data rusak (Not-A-Number). Blok if ini bertugas mendeteksi kerusakan tersebut dan membatalkan sisa kode di bawahnya dengan perintah return;.

2.6 Latihan

Kerjakan latihan berikut untuk menguji retensi konsep Anda secara mandiri.

- **Latihan 1 (Pemahaman Sistem ADC):** Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit".

Nilai maksimal ADC adalah 1023 karena menggunakan resolusi 10-bit. Dengan 10-bit, ADC memiliki $2^{10} = 1024$ tingkat nilai, yaitu dari 0 sampai 1023.

- **Latihan 2 (Implementasi Algoritma):** Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!

nilai ADC 0–1023 dapat diubah menjadi persentase 0–100% dengan rumus:

Persentase = (Nilai ADC / 1023) × 100

Perintah pembagian yang digunakan adalah:

```
persentase = (nilaiADC / 1023.0) * 100;
```

- **Latihan 3 (Analisis Rangkaian):** Di subbab 2.3.4 disebutkan sifat modul *Relay* yang terisolasi *Optocoupler*. Banyak *Relay* di pasaran yang bekerja secara Active-Low. Jika *Relay* Anda bertipe *Active-Low*, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke *Relay* jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`?

Jika *Relay* bertipe *Active-Low*, maka `digitalWrite(relayPin, HIGH)` membuat relay tidak aktif (OFF). Akibatnya, aktuator seperti lampu yang dikendalikan relay akan mati.

- **Latihan 4 (Pengembangan Logika):** Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok

kondisional if yang berfungsi menyalakan sebuah LED (D1) HANYA JIKA sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200)

LED D1 akan menyala hanya ketika nilai ADC LDR di bawah 200. Logikanya:

```
if (nilaiLDR < 200) {  
    digitalWrite(LED_PIN, HIGH);  
} else {  
    digitalWrite(LED_PIN, LOW);  
}
```

Artinya, jika $ADC < 200$, LED menyala karena kondisi dianggap gelap gulita. Jika $ADC \geq 200$, LED mati.

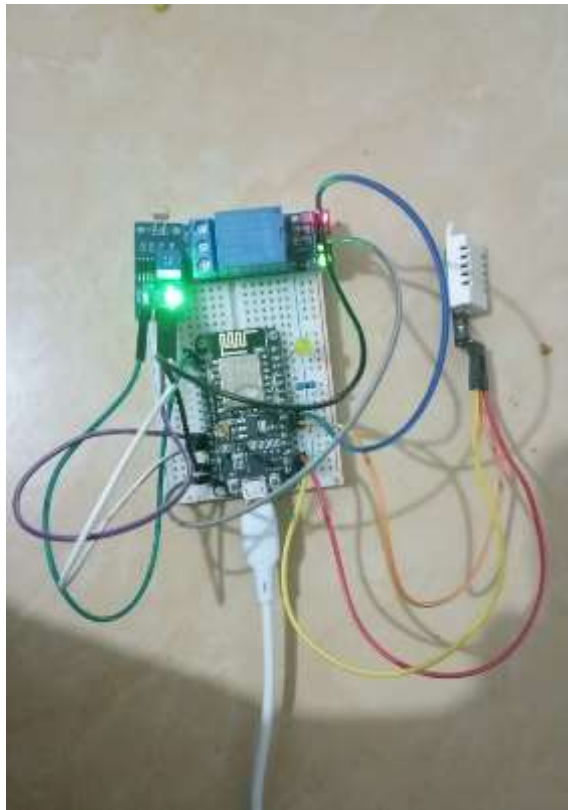
2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu

Sebuah sistem *Internet of Things (IoT)* harus memiliki kecerdasan dasar pada perangkat lapangannya (*Edge Computing*), sehingga apabila sewaktu-waktu koneksi internet terputus, perangkat masih dapat mengambil keputusan darurat berdasarkan kondisi lingkungan lokalnya.

Skenario Proyek (*Smart Warehouse*): Anda ditugaskan merancang *rule engine* aktuator mandiri. Sistem harus membaca Suhu (DHT) dan Intensitas Cahaya (LDR) secara bersamaan. Modul Relay 5V akan disimulasikan sebagai sakelar daya untuk pendingin ruangan (*Exhaust Fan*) atau sistem pencahayaan darurat.

- **Tujuan:** Mampu memadukan akuisisi multi-sensor (Analog LDR dan Digital DHT) dan menggunakan aktuator Relay untuk mengeksekusi keputusan terpadu berbasis kondisi/ambang batas.
- **Tingkat Kesulitan:** Menengah-Lanjut
- **Instruksi Perangkaian:**
 - i. Pertahankan sirkuit sensor LDR (di A0) dan DHT (di D7).
 - ii. Hubungkan pin VCC / +5V dari Modul Relay ke pin VIN atau VU NodeMCU (mengambil daya murni 5V dari USB). Jangan menyambungkan modul Relay 5V ke pin 3V3!
 - iii. Hubungkan pin GND Relay ke jalur GND bersama NodeMCU.
 - iv. Hubungkan pin IN / Signal Relay ke pin D6 (GPIO 12).

- v. *(Opsional sebagai indikator visual tambahan)*: Pasang sebuah LED beresistor 220 Ohm pada pin D1 (GPIO 5).
- Instruksi Pemrograman (Aturan/Rule Engine):
 - i. Konfigurasi pin Relay (D6) dan LED (D1) sebagai OUTPUT. (Kirimkan perintah mati di awal fungsi setup() untuk keamanan).
 - ii. Buatlah struktur logika perbandingan (if ... else) menggunakan operator logika OR (||).
 - iii. Aktifkan Relay dan nyalakan LED indikator JIKA kondisi lingkungan buruk berikut terpenuhi:
 - Suhu udara terbaca di atas 34 Celcius (Terlalu panas).
 - ATAU Nilai ADC LDR terbaca di bawah angka 300 (Terlalu gelap).
 - iv. Jika suhu normal dan ruangan cukup cahaya, pastikan Relay dan LED kembali mati.
 - v. Cetak status sistem ke Serial Monitor secara rutin (Contoh: "Kondisi Aman" / "Peringatan: Aktuator Aktif!").
- Kriteria Keberhasilan:
 - Sensor LDR dan DHT berhasil dibaca bersamaan dalam satu *loop* tanpa interupsi atau kegagalan pembacaan (*NaN*).
 - Logika kondisi OR beroperasi dengan presisi (cukup salah satu syarat buruk terpenuhi, aktuator menyala).
 - Relay bekerja selaras dan logis sesuai dengan sifat modul kelistrikan tersebut (pahami apakah modul Anda bersifat *Active-Low* atau *Active-High*).



Blink.ino

```
1  #include <DHT.h>
2
3  const byte dhtPin = 13;
4  #define DHTTYPE DHT11
5
6  DHT dht(dhtPin, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     dht.begin();
11     Serial.println("Memulai Sensor Lingkungan...");
12 }
13
14 void loop() {
15     delay(2500);
16
17     float temp = dht.readTemperature();
18     float hum = dht.readHumidity();
19
20     if (isnan(temp) || isnan(hum)) {
```

```
20   if (isnan(temp) || isnan(hum)) {  
21       Serial.println("Gagal membaca data dari sensor DHT!");  
22       return;  
23   }  
24  
25   Serial.print("Suhu: ");  
26   Serial.print(temp);  
27   Serial.print(" Celcius | Kelembapan: ");  
28   Serial.print(hum);  
29   Serial.println(" %");  
30 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

Suhu: 31.40 C | LDR: 757
Kondisi Aman

Suhu: 31.40 C | LDR: 756
Kondisi Aman
